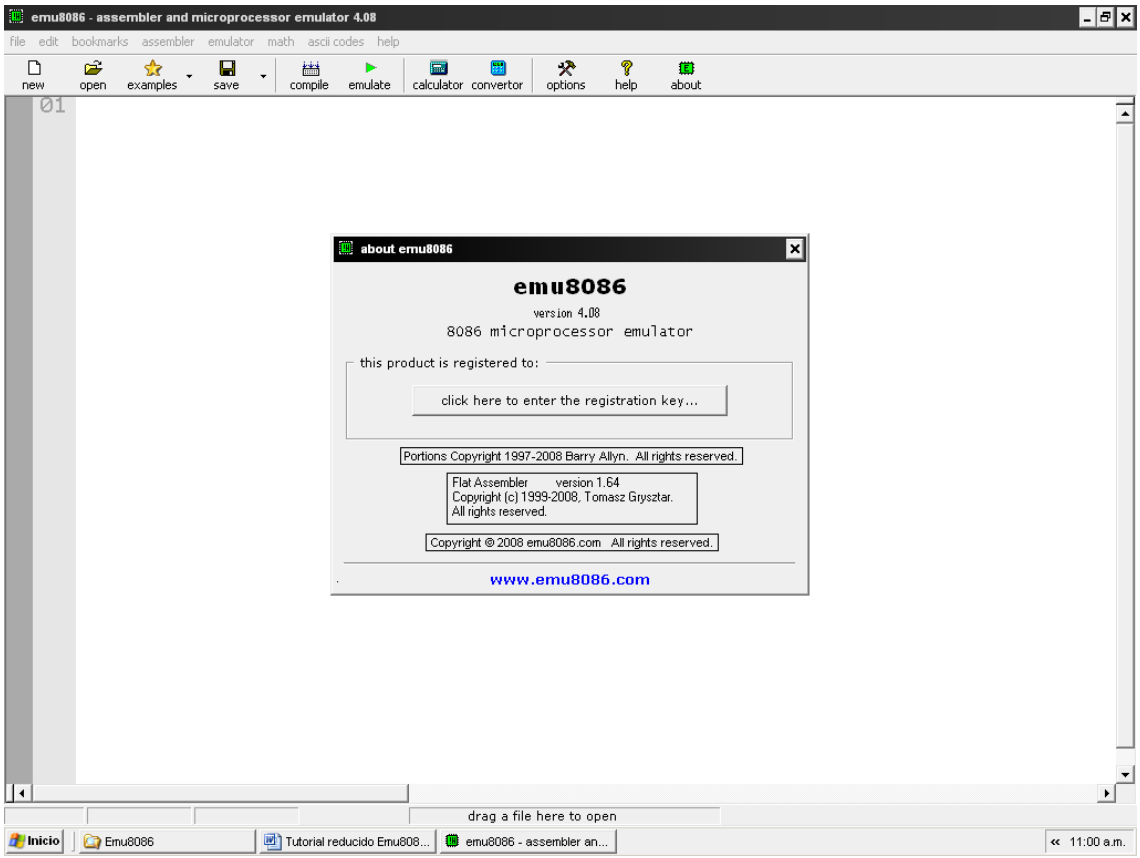
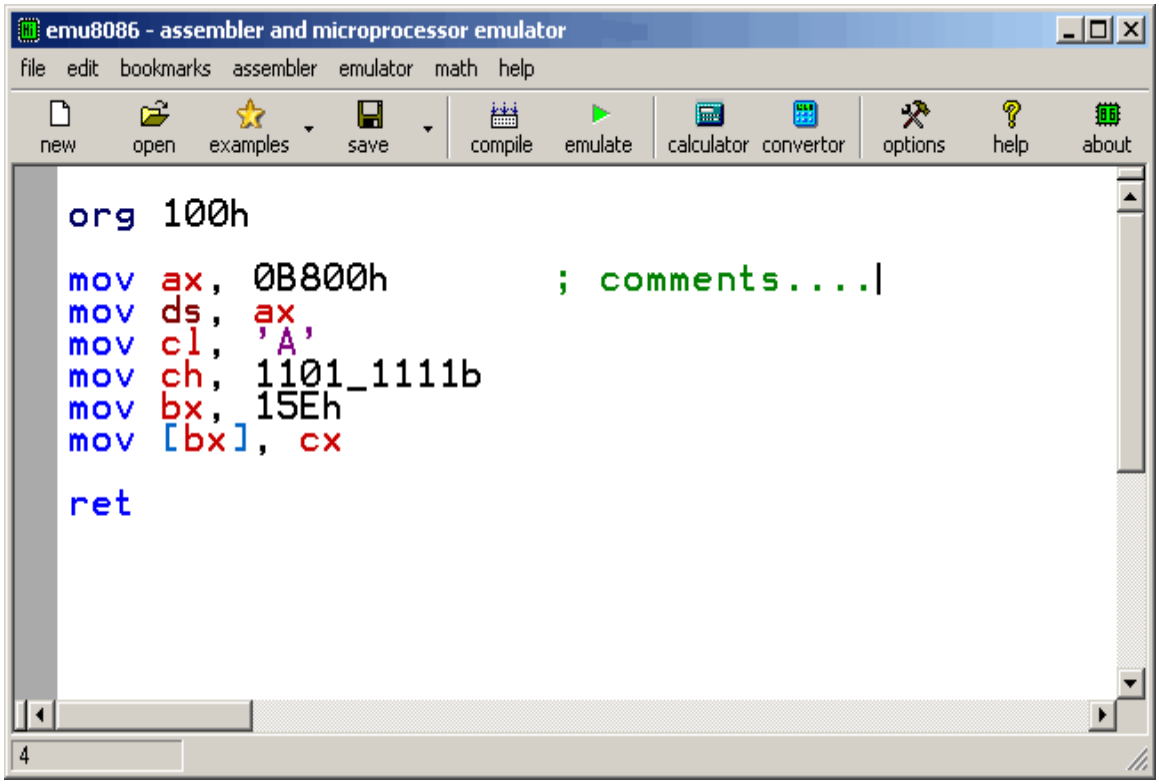
Tutorial Reducido Emu8086



1) compiling the assembly code



type your code inside the text area, and click **compile** button. you will be asked for a place where to save the compiled file.
after successful compilation you can click **emulate** button to load the compiled file in emulator.

---

the output file type directives:

```
#make_com#
#make_bin#
#make_boot#
#make_exe#
```

you can insert these directives in the source code to specify the required output type for the file. only if compiler cannot determine the output type automatically and it when it cannot find any of these directives it may ask you for *output type* before creating the file.

there is virtually no difference between how .com and .bin are assembled because these files are raw binary files, but .exe file has a special header in the beginning of the file that is used by the operating system to determine some properties of the executable file.

---

description of output file types:

- **#make_com#** - the oldest and the simplest format of an executable file, such files are loaded with 100h prefix (256 bytes). Select **Clean** from the **New** menu if you plan to compile a COM file. Compiler directive **ORG 100h** should be added before the code. Execution always starts from the first byte of the file. This file type is selected automatically if **org 100h** directive is found in the code.
  supported by DOS and Windows Command Prompt.

- **#make_exe#** - more advanced format of an executable file. not limited by size and number of segments. stack segment should be defined in the program. you may select **exe template** from the **new** menu in to create a simple exe program with pre-defined data, stack, and code segments. the entry point (where execution starts) is defined by a programmer. this file type is selected automatically if **stack** segment is found.
  supported by dos and windows command prompt.

- **#make_bin#** - a simple executable file. You can define the values of all registers, segment and offset for memory area where this file will be loaded. When loading "**MY.BIN**" file to emulator it will look for a "**MY.BINF**" file, and load "**MY.BIN**" file to location specified in "**MY.BINF**" file, registers are also set using information in that file (open this file in a text editor to edit or investigate).
  in case the emulator is not able to find "**MY.BINF**" file, current register values are used and "**MY.BIN**" file is loaded at current **CS:IP**.
  the execution starts from values in **CS:IP**.
  bin file type is not unique to the emulator, however the directives are unique and will not work if .bin file is executed outside of the emulator because their output is stored in a separate file independently from pure binary code.

  .BINF file is created automatically if assembler finds any of the following directives.

  these directives can be inserted into any part of the source code to preset registers or memory before starting the program's execution:

```
#make_bin#
#LOAD_SEGMENT=1234#
#LOAD_OFFSET=0000#
#AL=12#
#AH=34#
#BH=00#
#BL=00#
#CH=00#
#CL=00#
#DH=00#
#DL=00#
#DS=0000#
#ES=0000#
#SI=0000#
#DI=0000#
#BP=0000#
#CS=1234#
#IP=0000#
#SS=0000#
#SP=0000#
#MEM=0100:FFFE,00FF-0100:FF00,F4#
```

- all values must be in hexadecimal.

  when not specified these values are set by default:
  LOAD_SEGMENT = 0100
  LOAD_OFFSET = 0000
  CS = ES = SS = DS = 0100
  IP = 0000

  if **LOAD_SEGMENT** and **LOAD_OFFSET** are not defined, then **CS** and **IP** values are used and vice-versa.

  "#mem=..." directive can be used to write values to memory before program starts
  #MEM=nnnn,[bytestring]-nnnn:nnnn,[bytestring]#
  for example:
  #MEM=1000,01ABCDEF0122-0200,1233#
  all values are in hex, nnnn - for physical address, or (nnnn:nnnn) for logical address.
  - separates the entries. spaces are allowed inside.

  note: all values are in hex. hexadecimal suffix/prefix is not required. for each byte there must be exactly 2 characters, for example: 0A, 12 or 00.

  if none of the above directives directives are preset in source code, binf file is not created.
  when emulator loads .bin file without .binf file it will use

c:\emu8086\default.binf instead.
this also applies to any other files with extensions that are unfamiliar to the emulator.

- ---
- the format of a typical **".BINF**" file:

```
8000      ; load to segment.
0000      ; load to offset.
55       ;  AL
66       ;  AH
77       ;  BL
88       ;  BH
99       ;  CL
AA       ;  CH
BB       ;  DL
CC       ;  DH
DDEE     ;  DS
ABCD     ;  ES
EF12     ;  SI
3456     ;  DI
7890     ;  BP
8000     ;  CS
0000     ;  IP
C123     ;  SS
D123     ;  SP
```

- we can observe that first goes a number in hexadecimal form and then a comment.
  Comments are added just to make some order, when emulator loads a **BINF** file it does not care about comments it just looks for a values on specific lines, so line order is very important.
  NOTE: existing .binf file is automatically overwritten on re-compile.
- ---
- In case **load to offset** value is not zero (0000), **ORG ????h** should be added to the source of a **.BIN** file where **????h** is the *loading offset*, this should be done to allow compiler calculate correct addresses.

- **#make_boot#** - this type is a copy of the first track of a floppy disk (boot sector). the only difference from #make_bin# is that loading segment is predefined to 0000:7c00h (this value is written to accompanied .binf file). in fact you can use #make_bin# without any lack of performance, however to make correct test in emulator you will need to add these

directives: #cs=0# and #ip=7c00# - assembler writes these values into .binf file.
You can write a boot sector of a virtual floppy (FLOPPY_0) via menu in emulator:
[virtual drive] -> [write 512 bytes at 7c00 to boot sector]
first you should compile a **.bin** file and load it in emulator (see "**micro-os_loader.asm**" and "**micro-os_kernel.asm**" in c:\emu8086\examples for more information).

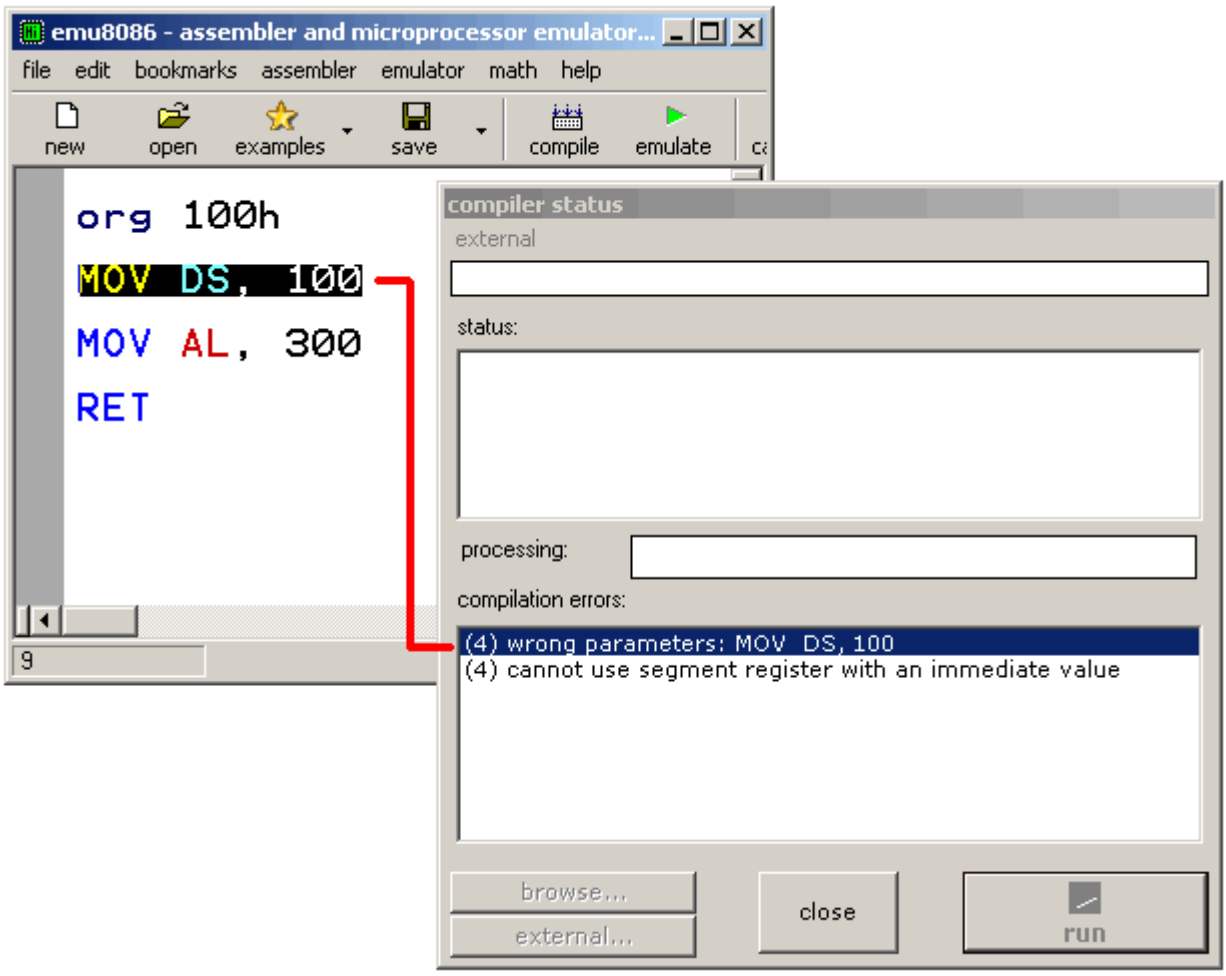
then select **[virtual drive] -> [boot from floppy]** menu to boot emulator from a virtual floppy.

then, if you are curious, you may write the same files to real floppy and boot your computer from it. you can use "writebin.asm" from c:\emu8086\examples\
micro-operating system does not have ms-dos/windows compatible boot sector, so it's better to use an empty floppy disk. refer to tutorial 11 for more information.
compiler directive **org 7c00h** should be added before the code, when computer starts it loads first track of a floppy disk at the address 0000:7c00.
the size of a **boot record** file should be less then 512 bytes (limited by the size of a disk sector).
execution always starts from the first byte of the file.
this file type is unique to emu8086 emulator.

---

error processing

assembly language compiler (or assembler) reports about errors in a separate information window:



MOV DS, 100 - is illegal instruction because segment registers cannot be set directly, general purpose register should be used, for example
MOV AX, 100
MOV DS, AX

MOV AL, 300 - is illegal instruction because **AL** register has only 8 bits, and thus maximum value for it is 255 (or 11111111b), and the minimum is -128.

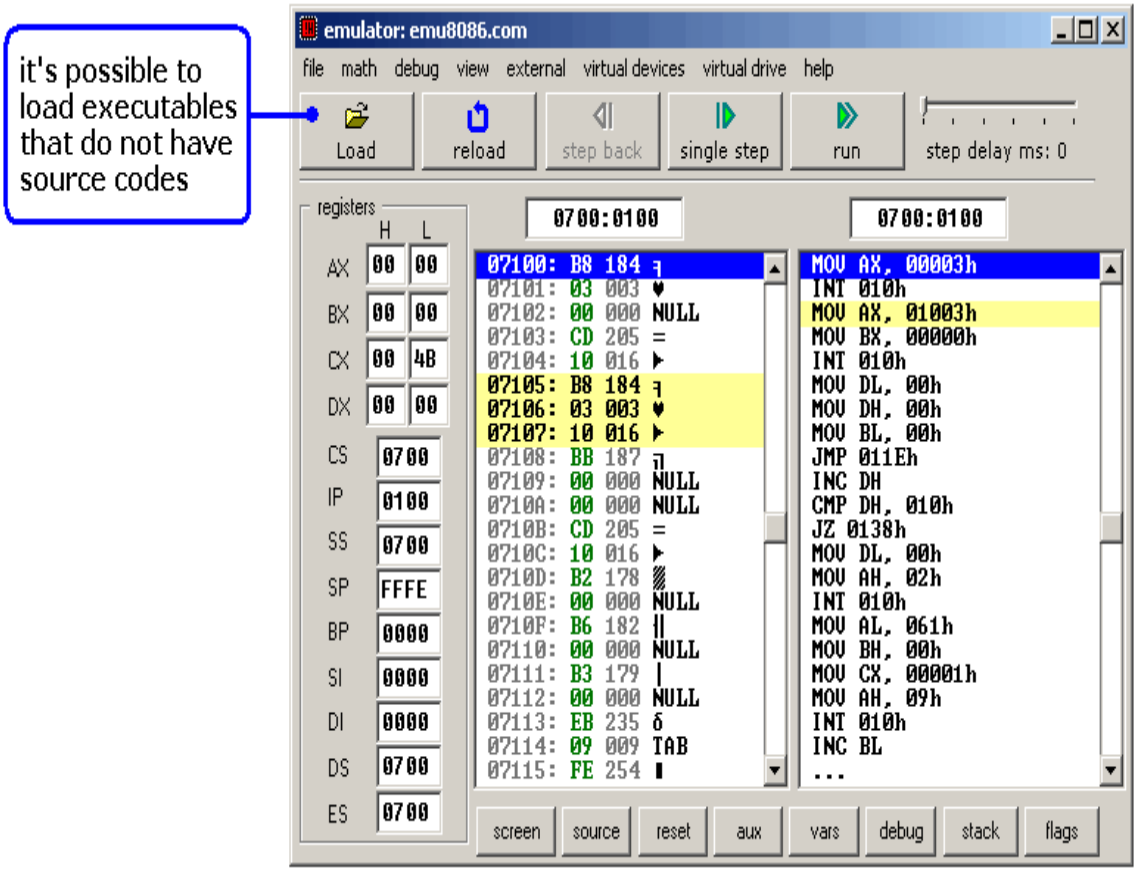
When saving an assembled file, compiler also saves 2 other files that are later used by the emulator to show original source code when you run the binary executable, and select corresponding lines. Very often the original code differs from the disabled code because there are no comments, no segment and no variable declarations. Compiler directives produce no binary code, but everything is converted to pure machine code. Sometimes a single original instruction is assembled into several machine code instructions, this is done mainly

for the compatibility with original 8086 microprocessor (for example **ROL AL, 5** is assembled into five sequential **ROL AL, 1** instructions).

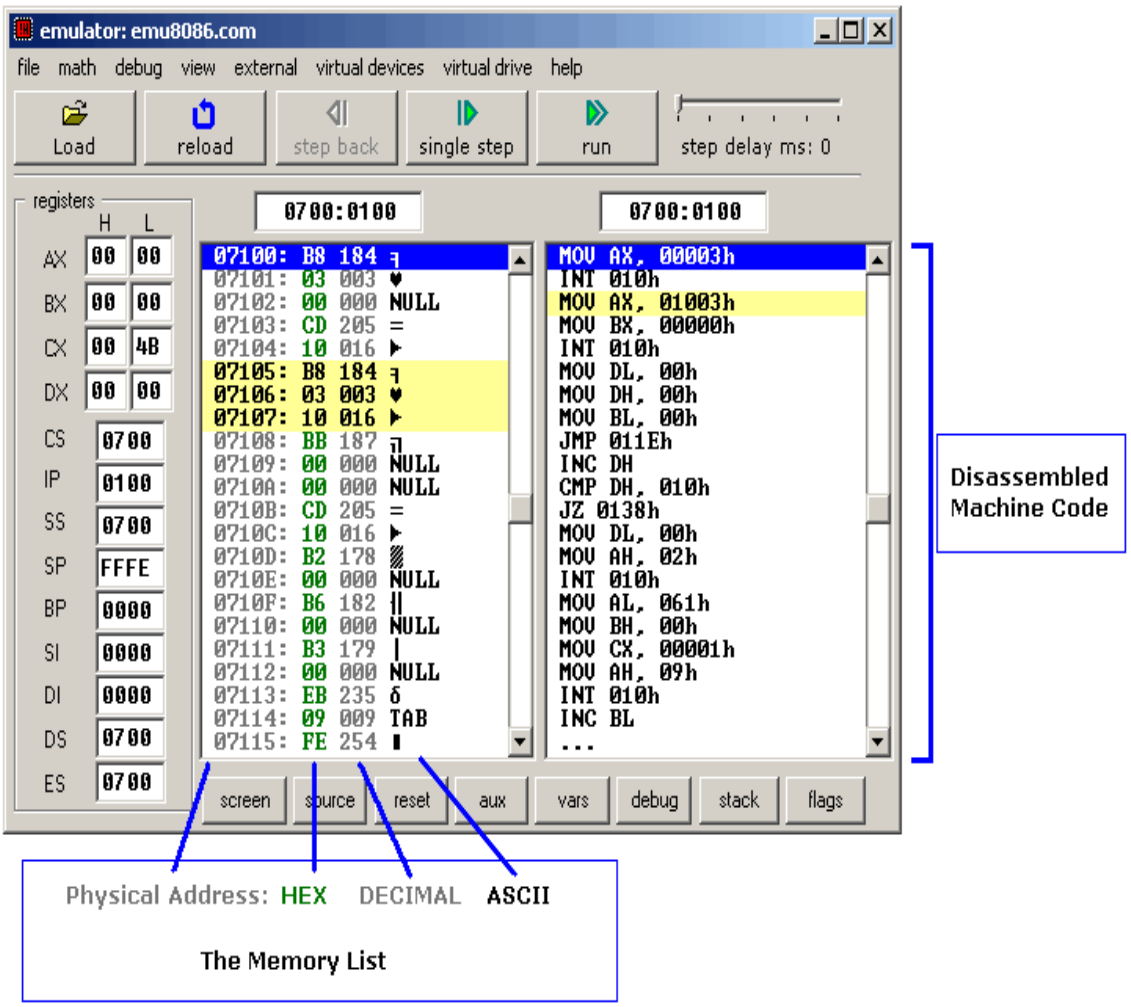
- ***.~asm** - this file contains the original source code that was used to make an executable file.
- ***.debug** - this file has information that enables the emulator select lines of original source code while running the machine code.
- ***.symbol** - symbol table, it contains information that enables to show the "variables" window. It is a plain text file, so you may view it in any text editor (including emu8086 source editor).
- ***.binf** - this ASCII file contains information that is used by emulator to load BIN file at specified location, and set register values prior execution; (created only if an executable is a BIN file).

2) Using the emulator

If you want to load your code into the emulator, just click "**Emulate**" button .
But you can also use emulator to load executables even if you don't have the original source code. Select **Show emulator** from the **Emulator** menu.



Try loading files from "**MyBuild**" folder. If there are no files in "**MyBuild**" folder return to source editor, select *Examples* from *File* menu, load any sample, compile it and then load into the emulator:



[**Single Step**] button executes instructions one by one stopping after each instruction.

[**Run**] button executes instructions one by one with delay set by **step delay** between instructions.

Double click on register text-boxes opens "**Extended Viewer**" window with value of that register converted to all possible forms. You can modify the value of the register directly in this window.

Double click on memory list item opens "**Extended Viewer**" with WORD value loaded from memory list at selected location. Less significant byte is at lower address: LOW BYTE is loaded from selected position and HIGH BYTE from next memory address. You can modify the value of the memory word directly in the "**Extended Viewer**" window,

You can modify the values of registers on runtime by typing over the existing values.

[**Flags**] button allows you to view and modify flags on runtime.